



Unidade 1 — Back-End e Front-End



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Explicar** o protocolo HTTP e seus métodos;
- **Identificar** os componentes de uma requisição e uma resposta HTTP;
- **Compreender** o conceito de API RESTful;
- **Aplicar** os métodos HTTP (GET, POST, PUT, DELETE) em exemplos práticos;
- **Analisar** a estrutura de uma comunicação cliente-servidor.

Taxonomia de Bloom: Explicar, Identificar, Compreender, Aplicar, Analisar.



O Protocolo HTTP

- **HTTP** = HyperText Transfer Protocol;
- Protocolo de **comunicação** entre cliente (navegador) e servidor;
- Baseado em **requisição** e **resposta**;
- É **stateless**: não mantém estado entre requisições;
- Opera, por padrão, na **porta 80** (HTTP) ou **443** (HTTPS).





Componentes da Requisição HTTP

```
POST /api/usuarios HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{
  "nome": "Maria Silva",
  "email": "maria@email.com"
}
```

Componente	Descrição	Exemplo
Método	Ação a ser executada	POST
URL/Path	Recurso acessado	/api/usuarios
Headers	Metadados da requisição	Content-Type, Authorization
Body	Dados enviados (quando aplicável)	JSON com nome e email



Métodos HTTP

Método	Ação	CRUD	Exemplo
GET	Ler/Listar	Read	<code>GET /api/usuarios</code>
POST	Criar	Create	<code>POST /api/usuarios</code>
PUT	Atualizar (completo)	Update	<code>PUT /api/usuarios/1</code>
PATCH	Atualizar (parcial)	Update	<code>PATCH /api/usuarios/1</code>
DELETE	Excluir	Delete	<code>DELETE /api/usuarios/1</code>

- **GET:** não altera dados, é **idempotente** (repetir retorna o mesmo resultado);
- **POST:** cria um novo recurso, **não** é idempotente;
- **PUT:** atualiza todo o recurso, é idempotente;
- **DELETE:** remove o recurso, é idempotente.



Componentes da Resposta HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 1,
  "nome": "Maria Silva",
  "email": "maria@email.com"
}
```

Componente	Descrição	Exemplo
Status Code	Resultado da operação	201 Created
Headers	Metadados da resposta	Content-Type
Body	Dados retornados	JSON com o usuário criado



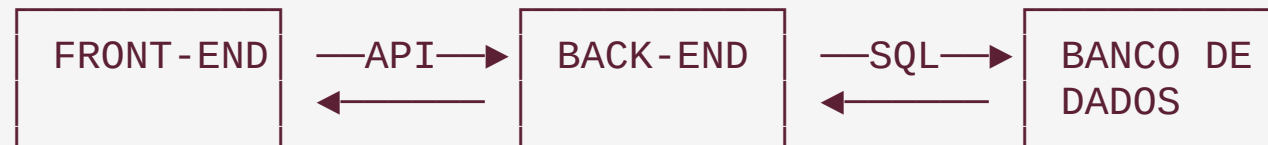
Status Codes HTTP

Faixa	Significado	Exemplos
1xx	Informacional	100 Continue
2xx	Sucesso	200 OK, 201 Created, 204 No Content
3xx	Redirecionamento	301 Moved, 304 Not Modified
4xx	Erro do cliente	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Erro do servidor	500 Internal Server Error, 503 Service Unavailable



O que é uma API?

- **API** = Application Programming Interface;
- É um **contrato** de comunicação entre sistemas;
- Define **endpoints** (URLs), **métodos** (HTTP) e **formatos** (JSON);
- O Front-End **consome** a API; o Back-End **expõe** a API.



Analogia: A API é como o cardápio de um restaurante. Ele lista o que está disponível (endpoints), como pedir (métodos) e o que esperar como resultado (formato).



API RESTful

- **REST** = Representational State Transfer;
- Estilo arquitetônico para APIs baseado em HTTP;
- Princípios fundamentais:

Princípio	Descrição
Stateless	Cada requisição contém todas as informações necessárias
Recursos	Tudo é identificado por uma URL (/api/usuarios/1)
Métodos HTTP	Ações são representadas pelos métodos (GET, POST, PUT, DELETE)
Representações	Dados são representados em JSON (ou XML)



Exemplo Prático: Requisição e Resposta

Front-End (JavaScript):

```
// Requisição GET para listar usuários
fetch('/api/usuarios')
  .then(response => response.json())
  .then(usuarios => {
    console.log(usuarios);
    // Atualizar a interface com os dados
  })
  .catch(error => console.error('Erro:', error));
```

Back-End (Express):

```
// Rota GET para listar usuários
app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});
```



Síntese da Aula

Conceitos principais:

1. **HTTP** é o protocolo de comunicação entre cliente e servidor;
2. Uma requisição HTTP possui **método**, **URL**, **headers** e **body**;
3. Os **métodos HTTP** (GET, POST, PUT, DELETE) correspondem às operações CRUD;
4. Uma **API RESTful** é uma interface baseada em HTTP e JSON;
5. O **fetch()** é o mecanismo do JavaScript para enviar requisições HTTP.

Pergunta reflexiva:

Por que o HTTP é stateless? Quais são as vantagens e desvantagens dessa característica?

Próxima aula: AJAX e comunicação assíncrona — Vamos aprofundar a comunicação sem recarregar a página.

